PLAN — Logging off Early

Role-aware log intelligence on SereneDB · National AI Hackathon 2026 · Track 04

Team: Arnie (`Arnie1x`) + Adnan (`AdnanSN`) — two people, one repo, `github.com/Arnie1x/exhibit` **Repo visibility:** private during the build, public at submission (the rules require a public repo; we do not want competitors reading our queries mid-build). **Written:** Saturday 17 October 2026, 10:30 CEST. **Deadline:** Sunday 18 October, 23:59 CET — **37 hours.**

> **How to use this document.** Sections 1–10 are the design; read them once, end to end.
>
> Section 11–12 are what you actually do, and when. Section 13 is tonight's Q&A. Section 16
>
> is the rule list for a coding agent — if you are Claude Code or Codex, read §16 and
>
> `docs/capabilities.md` first, then work through §12.
>
> **Every number in this document was measured on the live instance**, not estimated. Re-run
>
> `make smoke` to see them again. Where a figure comes from the organisers rather than from
>
> us, it says so.

---

1. What changed this morning

Two documents from the SchubKI / SereneDB team landed in Discord and they **overturn two assumptions** we had both been planning around:

We assumed	Actually
There is a supplied loader, schema and team-ownership table to build on	**"There is no pre-built loader, schema or ownership table, and there are no shared connection details."** We build all three, and we invent the ownership table ourselves
There is a shared SereneDB instance we might lack DDL rights on	There is none. Everyone runs their own locally — so **we own the machine completely**

Three further facts worth having:

- **The docs are searchable from inside the database**: `SELECT * FROM sdb_docs.search('inverted index', 3)` returns `path, title, breadcrumb, snippet, score` . An agent can look up SereneDB's documentation **offline, over SQL**, with no network. There is also a `docs` table with its own `docs_fts` index.
- **The docs MCP is one command** — `claude mcp add --transport http serenedb-docs https://api.serenedb.com/mcp` , or `codex mcp add serenedb-docs --url https://api.serenedb.com/mcp` . Three tools: `search_docs` , `read_section` , `docs_health` . **Do this first; it is the organisers' step zero.**
- **The huge-datasets award is genuinely optional** and needs no download: index a *view* over remote Parquet (`read_parquet` over HTTP/S3/HF), and only the index is stored locally. Refresh with `REINDEX INDEX` .

Consequence for us: the riskiest dependency in both plans — "wait for the kit, hope we can create tables" — is gone. Our local instance is already loaded, derived, indexed and measured. We are ahead of where either plan assumed we would be at this hour, and we should treat the next 37 hours as *build and validate*, not *set up*.

---

2. The thesis, in one line

> **SQL computes. The LLM explains.**

The model never invents a number. Every figure in every answer is produced by a query, carried in a result envelope, and cited by evidence ID. An automated check rejects any answer whose numbers do not appear in the tool output.

Three rules follow, and everything else in this document is downstream of them:

1. `@@` **only works against an inverted-indexed relation.** Query the index relation *by name* (`bgl_idx`, `os_idx`); aggregate on the tables (`bgl_ev`, `os_ev`). A plain table with an index attached to it still fails. This is the single most likely hour-killer in the project.
2. **Count incidents, not lines.** Lines are evidence; an incident is signature × blast radius × persistence. §9 has the measured proof.
3. **Roles differ in what they can *see*, enforced in the database** — separate views, separate GRANTs, `SET ROLE`. Never in the system prompt alone. The manager's view is body-free, so a leaked prompt cannot leak log text.

---

3. What we run on (all verified today)

The box: `hermes` — Ubuntu 26.04, **aarch64**, 2 vCPU, 12 GB RAM, no CUDA, ~71 GB free.

```
# arm64 caveat: 26.10.x tags are amd64-only. 26.09.1.3 is the newest multi-arch tag.
docker run -d --name serenedb -p 127.0.0.1:7890:7890 \
  -v serenedb-data:/var/lib/serenedb -v "$PWD/data":/data serenedb/serenedb:26.09.1.3
psql -h 127.0.0.1 -p 7890 -U postgres      # no credentials by default
```

Banner: `PostgreSQL 18.3 (SereneDB 26.09.1.3)`, native arm64.

Bound to loopback only, deliberately. Docker's port publishing bypasses `ufw` entirely, and the engine runs with trust auth. A teammate connects over Tailscale (`psql -h 100.108.122.19 -p 7890 -U postgres`), never over the public IP.

Measured timings (these set the whole schedule):

Step	Time
Derive `bgl_ev` (4.75M rows) + `os_ev` (208k) from Parquet	**14 s**
Build `bgl_idx` (15 fields)	**5.2 s**
Build `os_idx`	**0.4 s**
Full rebuild (`make bootstrap`)	**~20 s**
Read a 1-minute slice of a *remote* 1B-row Parquet over HTTPS	**0.8 s**

A full rebuild in twenty seconds is a strategic asset: **we can afford to rebuild the world between attempts.** Never hand-patch state; re-run the scripts.

UI (optional): SereneUI on port 6543, `http://100.108.122.19:6543`. Nice for screenshots in the video. It is a convenience, not a deliverable — do not let it eat build time.

---

4. The corpus, and what is actually in it

Both datasets from Zenodo record 8196385 (~770 MB), parsed by `scripts/parse.py` into Parquet. **BGL loses zero lines** (4,747,963 of 4,747,963); OpenStack loses 8 of 207,820.

	BGL	OpenStack
Rows	**4,747,963**	**207,812**
Span	2005-06-03 → 2006-01-04	2017-05-14 19:39 → 2017-05-17 12:02
Alert lines (`label <> '-'`)	**348,460**	—
Distinct nodes (id string)	69,252	—
Components	10	—
Error level lines	ERROR 112,355 · FATAL 855,501	ERROR 186 · CRITICAL 2

BGL has four record shapes, which is why a naive parser silently drops ~0.7% of the file. The anchor is the `<TYPE> <COMPONENT> <LEVEL>` triple, where TYPE is `RAS` **or** `NULL` (100,141 lines are `NULL <COMPONENT> <LEVEL>`). There are also empty-message 9-field lines and multi-line entries where the message text comes *before* the anchor. All four are handled — `docs/data-notes.md` has the details, and any future edit to `parse.py` must keep `4,747,963 / 4,747,963` as the parity check.

The five findings that shape this build

1. BGL's `-` label does not mean healthy. 84,253 lines carry the "normal" label while containing `idoproxydb hit ASSERT condition: ASSERT expression=0`. Building the recurring-problems view on the label column undercounts faults by ~24%. Build it on **message signatures**.

2. Line count ranks incidents backwards. The money comparison, both halves measured:

Event	Lines	Nodes
Burst, 2005-06-11 17:17 → 22:59	**136,209**	**1**
Fleet event, 2005-06-14	**49,154**	**22,529**

39% of all alert lines in the corpus come from one node in under six hours. Ranking by volume puts a single sick machine on top and buries a fleet-wide event that touched 22,529 nodes. There is a third day, 2005-06-13, with 17,409 lines across 16,644 nodes.

3. Ranking by *episodes* inverts the ranking again. At a 5-minute gap the leaders are not the volume leaders:

Signature	Lines	Episodes
`data TLB error interrupt`	152,734	117
`Lustre mount FAILED : bglio{N} : point /p/gb{N}`	26,426	**3,667**
`ciod: Error reading message prefix on CioStream socket`	26,546	**3,638**
`rts: kernel terminated for reason {N}`	18,587	2,641

4. OpenStack's entire error surface is one bug, repeated eight times. `oslo_service.periodic_task` emits **184 ERROR lines**, all from `nova.virt.libvirt.imagecache` → `qemu_img_info` → `InvalidDiskInfo`. There are exactly **8** `Traceback (most recent call last):` headers. The same root cause produces **3,059 WARNING lines** (`Unknown base file: /var/lib/nova/instances/_base/...`). 188 error lines ≈ **8 incidents of 1 bug.**

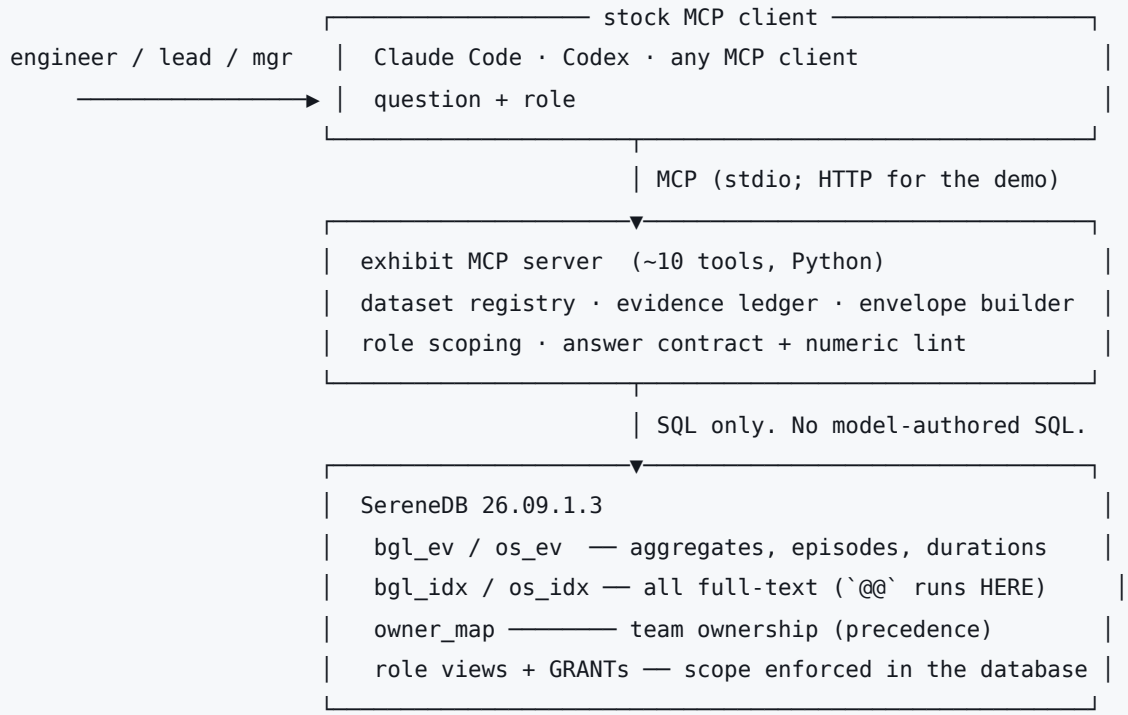
And one signal that volume hides: `keystonemiddleware.auth_token` emits **2 CRITICAL** lines — the only CRITICAL in the entire corpus — plus 2 ERROR and 2 WARNING. Six lines. A line-count leaderboard will never show them. Our demo will.

5. The labelled anomalies are only visible through durations. The four VMs in `anomaly_labels.txt` produce 25 lines each, **all at INFO level**, and aggregate latency is identical between the normal and abnormal files. Sorting instances by instance-lifecycle duration puts **all four labelled anomalies in positions 1–4** (spawn 50.72 s, 41.81 s, 31.27 s, 29.46 s), with the first unlabelled instance at position 5. No level filter or latency percentile finds them at all.

Also worth knowing: **67,363 lines (32%) are one poller** (`GET /v2/{tenant}/servers/detail`), and 3,969 of the API calls are 404s. Top-message rankings surface the poller, not the problems.

---

5. Architecture



Five layers, each with one job:

1. **Data layer** — `sql/10_derive.sql`, `sql/20_indexes.sql`. Raw Parquet → derived tables → index relations. Idempotent, 20 s.
2. **Semantic layer** — `sql/30_views.sql`. Signatures, episodes, incidents, durations, role views. This is where domain knowledge lives; the tools are thin on top of it.
3. **Tool layer** — the MCP server. Ten tools, each returning the **envelope**: `{rows, query, filters, scope, total_matches, returned, truncated, evidence_id}`. The envelope is what makes "show your work" automatic and what makes the numeric lint possible.
4. **Agent layer** — a stock MCP client. We do **not** build an agent framework. The role is a system prompt *plus* a database identity.
5. **Presentation layer** — the answer contract. Scope → Observed → Hypothesis → Not established → Next steps → Evidence, always in that shape.

The dataset registry (`datasets.py`) maps logical names to physical columns and lives in front of everything. It exists so that a column rename never touches a tool, and so the same tool code works against BGL and OpenStack without branching in twenty places.

---

6. Data model

```
bgl_ev(id, ts, alert_category, is_alert, node, node_kind, rack, midplane, nodecard,
       component, level, level_rank, template_id, team, content)
os_ev (id, ts, source_file, service, logger, level, pid, req_id, user_id, tenant_id,
       instance_id, http_method, endpoint, status, resp_len, latency_s, spawn_s,
       build_s, team, content)
owner_map(dataset, key_type, key_value, team)          -- 23 rows
```

Sentinels, never NULL, in every facet or keyword column: `'UNKNOWN'`, `'NONE'`, `'UNOWNED'`, `'{n}'`. The term-dictionary operators need `NOT NULL` to group correctly, and a NULL that becomes a dropped row is a silently wrong number.

Ownership precedence: `logger` > `service` > `component`. An OpenStack logger name is more specific than the service it belongs to, so it wins. Keys that match nothing stay visible as `UNOWNED` rather than being dropped. Coverage after the join: **100% of both corpora**.

Team	BGL lines
compute-platform	4,325,112
workload-runtime	228,536
observability	99,064
storage-platform	88,930
site-infrastructure	5,148
network-fabric	1,173

That table is itself a demo point: **one owning team holds 91% of the corpus**, because the `KERNEL` component alone is 4,324,967 lines. A lead who sees this table learns something a lead who sees a total does not.

`template_id` is `'UNKNOWN'` throughout: no template IDs are supplied. Say so rather than inventing them.

---

7. The tool catalogue

Core ten. Each returns the envelope; each has a constraint it must respect.

Tool	Returns	Must respect
`describe_data`	datasets, scopes, column meaning, ownership	The only tool allowed to explain the schema; answers "what can you see?"
`search_logs`	matching lines with `ts_highlight` snippets	Searches `bgl_idx` / `os_idx`, never the tables
`count_events`	counts by signature/level/component/team	Returns total **and** returned **and** truncated
`top_signatures`	ranked signatures	Always shows lines **and** episodes **and** nodes side by side
`trend`	buckets over time (`date_bin`)	Anchored to the dataset's own last timestamp, not today
`durations`	lifecycle outliers per instance	The only path to OpenStack's labelled anomalies
`ownership`	team ↔ component/service mapping	Joins `owner_map` ; exposes UNOWNED explicitly
`nearby_events`	what else happened in the ±N minutes	Uses `id` ranges on the index relation
`what_changed`	significant terms, foreground vs baseline	`ts_dict_agg` / `ts_dict_count` , lift over chance
`episodes`	incidents, not lines	Coalescing gap is a **parameter** (1/5/60 min), stated in the answer

Optional five, only after the MVP gate: `trace_context` , `drop_index` / `create_index` (probe-gated, see §14), `validate` (hidden ground truth), `explain_query` .

Answer contract, enforced in code:

`Scope` → `Observed` (numbers, each with an evidence ID) → `Hypothesis` (labelled) → `Not established` (explicit gaps) → `Next steps` (each citing an evidence ID) → `Evidence` (the query).

Role playbooks — three different evidence paths, not three lengths:

- **engineer** → raw lines, highlighted snippets, timeline, neighbour events, the query itself. Granular, technical, no summarising away the evidence.
- **team_lead** → aggregated signatures grouped by owning **team**, backlog shape, what changed, where the work sits. No individual log lines.
- **manager** → signature level only, prioritisation by **blast radius and persistence**, and `Not established` for anything about money, customers or downtime. No component names, no raw text.

The manager lint rejects `€` , `$` , `downtime` , `customers` , `revenue` , `SLA` automatically. That guard exists because the brief explicitly warns against inventing business impact, and a rule in a prompt is a rule a model can be talked out of.

---

8. Episode model — the heart of the demo

An **episode** is a contiguous run of one signature in one failure domain, ended by a gap. A **failure domain** is a midplane. The gap is a parameter.

Measured, at a 5-minute gap:

	Count
Alert lines	**348,460**
Episodes, per signature × midplane	**19,250**
Episodes, per signature only	**1,377**

Line counts overstate the incident count by **18×** (failure-domain) or **253×** (global) — whichever granularity you pick, volume is the wrong ranking. The `episodes` tool takes the gap as a parameter and **states which value it used in the answer**, because 132 / 117 / 57 episodes for the same signature at 1 / 5 / 60 minutes is itself a finding worth showing.

---

9. The two worked investigations

These are the demo. Both are already reproduced end to end on the real corpus — they are not aspirational.

W1 — "Which problem should we fix first?" (BGL, engineer → lead → manager)

The question is deliberately the one every persona would ask. The three answers use different evidence:

- *engineer*: `top_signatures` returns `data TLB error interrupt` with 152,734 lines — and `search_logs` shows it is one node emitting 136,209 lines in 5h43m.
- *lead*: grouped by team, 91% of alert volume belongs to `compute-platform`; but by episodes, the recurring dull ache is `ciod` and `Lustre mount FAILED`, thousands of small incidents across eight nodes.
- *manager*: 348,460 alert lines → 19,250 incidents → ranked by blast radius and persistence, with `Not established` for cost. The recommendation is a statement about *what to fix first and why*, not a count.

W2 — "Were the instances that were supposed to be slow actually slow?" (OpenStack)

- *engineer*: latency percentiles are identical between the normal and abnormal files (mean 0.234 s both). Nothing to see.
- *lead*: `durations` ranks instances by lifecycle duration — the four labelled anomalies occupy positions 1–4.
- *manager*: eight incidents of one imagecache bug; six lines of CRITICAL auth failures that no volume ranking would surface.

Hidden validation (the credibility device). `anomaly_labels.txt` and BGL's `label` column are used by a **separate script** that the agent cannot call and cannot read. It reports recall (4 of 4) and the near-miss (position 5 is unlabelled) outside the agent's context. We disclose in the video that ground truth exists and that we kept it out of the agent's reach.

---

10. Scope guards

Not building: a custom agent framework (use a stock MCP client), a web GUI, free-form SQL generation from user text, model training or fine-tuning, a third dataset, anything that cannot be traced to a rubric line. Every one of these has killed hackathon teams with a better idea and less time.

Bonus work happens only after the MVP gate, in this priority order: MCP-server packaging for judges → the evaluation harness → the huge-datasets bench → workspace indexes (probe-gated, §14) → an

`investigations/` write-up of anything that did not fit the demo.

---

11. Schedule, mapped to the real clock

It is **10:30 Saturday**. The deadline is **23:59 Sunday**. Organiser advice: pick one approach by Saturday morning (done), and **sleep Saturday night**.

When	What	Gate — do not pass without this
Sat 10:30–12:00	Stage 0–1: probe + data layer. **Already done** — `make bootstrap smoke` prints the headline numbers on this box	`make bootstrap` < 60 s; `make smoke` prints 4,747,963 / 207,812 and 348,460 → 19,250
Sat 12:00–14:00	Stage 2: MCP server skeleton + `describe_data`, `search_logs`, `count_events` against the live DB	A stock client lists three tools and answers one question with an envelope
Sat 14:00–17:00	Stage 3: `top_signatures`, `trend`, `episodes`, `durations`, `ownership` — W1 and W2 runnable as queries	`make demo` prints both investigations, numbers matching §9
Sat 17:00–18:00	**Freeze for the Q&A.** Write the three questions you will ask (below) and rehearse the 10 minutes	You can explain W1 in 90 seconds without notes
Sat 18:00–20:00	**Per-team Q&A with the challenge owners** (10 min slot)	Answers written down and folded back into this plan; if a rubric reading changes, note it tonight
Sat 20:00–24:00	Stage 4: the three roles + answer contract + numeric lint + manager lint	Same question, three answers, three evidence paths — visibly different, not longer
Sun 00:00–08:00	**Sleep. Non-negotiable.** The plan assumes it	—
Sun 08:00–12:00	Stage 5: validation (hidden ground truth), the evaluation harness, fix what it finds	Eval score printed; every number in the README reproducible
Sun 12:00–15:00	Stage 6: README, `make clean-check`, slides, the disclosure section	A clean clone runs `make bootstrap smoke` on a machine that has never seen the project
Sun 15:00–19:00	Stage 7: **record the video** (5 min, per the ceremony notes — see §13 Q3), rehearse twice	Video plays end to end without a stall and shows a real query returning a real number
Sun 19:00–22:00	Stage 8: submission checklist, flip the repo public, upload	§15 complete
Sun 22:00–23:59	Buffer. Something will go wrong; this is the hour it goes wrong in	—

Cut lines, in order, if you fall behind: workspace-index bench → `trace_context` and `explain_query` → the eval harness (keep a manual 12-question list instead) → the huge-datasets bench → SereneUI screenshots. **Never cut:** the evidence ledger, numeric verification, W1/W2, the README quickstart, the rehearsals.

Risk register: the `@@`-on-the-wrong-relation mistake (§2 rule 1 — the most likely hour-killer); dictionary syntax drift (§16); MCP client wiring, which always takes longer than expected (timebox the *first* end-to-end call to one

hour on Saturday afternoon and freeze the client choice when it works); scope creep from bonus work; and the multi-line Python tracebacks being counted as several incidents (already handled by the signature mask).

---

12. Work split — two people, three streams

Three streams, two people: the SQL/data stream and the tool/agent stream run in parallel from this hour, and the demo stream is a **shared two-hour block each day**, not a solo stage.

Who	Owns	Stages
Arnie	MCP server, dataset registry, evidence ledger, tool contracts, role prompts + role views, answer lint, packaging	2, 4, and the verify half of 5
Adnan	Probe matrix, derive/index, views, ownership map, golden SQL, **independent verification of every number in this plan**, huge-datasets bench	0, 1, 3, and the ground-truth half of 5
Both	W1/W2, slides, video, README, the Q&A	5–8

The interface contract — agreed, so neither of you waits for the other:

1. Pattern B: plain derived tables + index relations, searches always target `*_idx` by name.
2. Adnan's schema and naming win: `bgl_ev`, `os_ev`, `owner_map`, `bgl_idx`, `os_idx`. They are in this repo and already loaded.
3. Arnie never writes SQL against column names directly — everything goes through `datasets.py`.
4. Any number quoted in a document, slide or video must be reproducible by `make smoke` or `make demo`. If you cannot point at the query, it does not go on stage.

Do this in the next two hours:

- **Adnan:** run `make probe`, then read `docs/capabilities.md` and try to **break** it — re-measure the partial-index row and the dictionary row. Independently confirm 348,460 → 19,250 and the 136,209-on-one-node figure. Then port `sql/50_roles.sql` (it still references the old flat tables) and extend `os_sig` with anything W2 needs.
- **Arnie:** get one real end-to-end call working — stock client → MCP server → `bgl_idx` → envelope → cited answer. Not five tools; **one**, and ugly is fine. Then the ledger.
- **Both:** in the Q&A, ask the questions in §13; write the answers into this file the same night.

---

13. Tonight's Q&A — 10 minutes, use them

Do not present. Ask, listen, write. You have ten minutes with the people who wrote the brief and the rubric; a question answered tonight is worth more than two hours of guessing, and there is exactly one slot.

Ask these, in this order:

1. **The brief says "twelve labelled alert categories"; BGL's first column has 42 distinct values (41 plus `-`). Which is authoritative for scoring?** *Measured: 42.*
2. **The brief says "five severity levels"; we measure six distinct values** (INFO 3,735,823 · FATAL 855,501 · ERROR 112,355 · WARNING 23,357 · SEVERE 19,213 · FAILURE 1,714 — no `Kill` in the derived set).

Measured: 6.

3. **Is the demo a 3-minute or a 5-minute slot?** The website says 3; the ceremony notes say 5. *Plan for 5, prepare a 3-minute cut.*
4. **May we use the supplied label column and `anomaly_labels.txt` as ground truth for a validation section that the agent cannot reach?** We do this and disclose it; confirm it is acceptable.
5. **For the ownership table we invent: is per-component/per-service granularity what you expect, or do you want end-to-end ownership of a request path?** We map logger > service > component and sit at 100% coverage.
6. **How will the judges actually connect to a submitted MCP server?** Which client, which command, what timeout? This determines how much packaging we do on Sunday.
7. **Manager-level answers: how precise should prioritisation be?** A ranked list with reasons, or a single recommendation with a confidence statement?
8. **Is a private repo during the build acceptable if it is public at submission?** (Also ask this in Discord in case the room is short.)

The 10-minute script: 30 seconds of introduction ("two MSc students, we built X") → then the **money comparison** from §4 finding 2 on screen (136,209 lines on one node vs 49,154 lines on 22,529 nodes) → then your questions. The comparison buys credibility in ten seconds and makes everyone in the room want to help you.

---

14. Bonus ladder — priority ordered, probe-gated

1. **MCP-server packaging** (2 h) — four connection paths, generated tool docs, a `.mcp.json`. Judged separately as *Best MCP Server*; free upside on the main score, because it is also how the judges experience the main deliverable.
2. **Evaluation harness** (2 h) — 12 questions, tool-selection accuracy and numeric accuracy scored against golden SQL. This is how Stage 5's 25% gets a number instead of an adjective. Arnie's lane.
3. **Huge-datasets bench** (2 h) — **the documented mechanism is a view over remote Parquet with an inverted index on the view, refreshed with `REINDEX INDEX`**, and only the index is local. Start with a one-minute slice (0.8 s to read), measure rows/second of build, extrapolate, and **report how much data you actually queried and how long it took** — the organiser Q&A asks for exactly that. Nothing must be downloaded.
4. **Workspace indexes — DO NOT BUDGET TIME YET.** Measured today: a partial inverted index shows **no effect** — naming it returns the full count (121,498) where the same predicate against the table returns 0, the planner does not select it (`EXPLAIN` → `Index: bgl`, `ts` as a *Column Filter*), and there is no timing difference (4.19 ms vs 3.99 ms). Re-probe first; if it still shows nothing, say so plainly in the video. The brief's bonus explicitly welcomes the negative result, and a team that measured and reported "this did not pay off" looks better than a team that claims a speedup it cannot show.
5. **`investigations/` write-up** (1 h) — anything real we found that did not fit the demo.

---

15. Submission checklist

☐ Public repo (flip from private at submission), MIT licence, README that a judge can follow cold

☐ `make bootstrap smoke` passes from a clean clone — **verify on a machine that never saw the project** (`make clean-check`)

☐ Schema, loading SQL, ownership table and scripts all in the repo (the organiser guidance asks for exactly these)

☐ README documents: how to start SereneDB, how to load BGL and OpenStack, how to start the agent workflow

☐ Disclosure section: models and tools used, and what was built during the window vs what existed before

☐ Video ≤ 5 min (with a 3-min cut prepared), real queries visible, real numbers on screen

☐ Slides

☐ SereneDB starred on GitHub (a listed prerequisite — do it now, it takes ten seconds)

☐ Docs MCP connected in the coding client (step zero from the organisers)

☐ `docs/capabilities.md` current — every claim in the video traceable to a measurement

---

16. Rules for coding agents

> The organisers' guidance is explicit: *"SereneDB is designed to be explored with AI agents, so we encourage you to use them heavily."* Connect the docs MCP before writing SQL, and prefer asking the database (`sdb_docs.search`) over guessing.

Rules that break builds:

1. `@@` runs **only** against an index relation: `FROM bgl_idx WHERE content @@ '…'`. Never against `bgl_ev`. A plain table with an index attached still fails with `TSQUERY expression evaluated outside an @@ match against an inverted-indexed column`.
2. Dictionary syntax is the parenthesised form: `CREATE TEXT SEARCH DICTIONARY log_text (template = 'text', locale = 'en_US.UTF-8', case = 'lower', stemming = false, accent = false, frequency = true, position = true)`. The `AS split_text(…)` form is a **syntax error**.
3. **SQL computes, the LLM explains.** No number may originate in the model. If you cannot cite a row, you cannot claim the number.
4. `VARCHAR(n)` is accepted and **not enforced** — never use it for validation.
5. `to_timestamp(text)`, `to_char`, `to_date` do not exist. Use `::timestamp`, `strptime`.
6. `regexp_extract` returns `''`, never NULL — always `nullif(x, '')`.
7. Timestamps arrive as VARCHAR: cast them. `date_trunc` needs `::timestamptz`.
8. Never support a count with `LIKE '%term%'` — tokenization differs, and `@@ 'base'` gives 21,212 while the older non-alpha index gives 45,306 and `ILIKE` gives 45,351. **Always state the operator you used.** See `docs/capabilities.md` §5.
9. `ts_offsets` / `ts_highlight` need an `@@` in the same sub-query.
10. There is no index-size metric: `sdb_metrics` reports connection/refresh/compaction counters, `sdb_indexes()` has no size columns, `pg_class` gives `relpages = 0`. Measure wall-clock and the data directory.

Conventions: Python 3.11+ with `uv`; SQL is idempotent (`DROP … IF EXISTS`); no secrets in the repo; local SereneDB on loopback with trust auth; conventional commits; one stage per branch at most; every claim in the README reproducible from a clean checkout.

---

17. Appendix

Verification, in one line each — re-runnable with `make smoke`:

```
BGL 4,747,963 rows · OpenStack 207,812 rows · span 2005-06-03 → 2006-01-04
alert lines 348,460 → 19,250 failure-domain episodes → 1,377 global episodes
136,209 lines on 1 node (2005-06-11)  vs  49,154 lines on 22,529 nodes (2005-06-14)
84,253 fault lines carrying the "normal" label
OpenStack: 188 error lines = 8 tracebacks of one bug + 6 CRITICAL auth lines
4 of 4 labelled anomalies in the top 4 by lifecycle duration
ownership coverage 100% · derive 14 s · index 5.6 s
```

Datasets: Loghub — BGL and OpenStack, Zenodo record 8196385. **Engine:** SereneDB 26.09.1.3 (Apache-2.0). **Disclosure:** the design and the code in this repo were produced during the build window with AI assistance (Hermes Agent / Claude Code as coding agents, SereneDB docs MCP for API facts). No pre-existing code from outside the window is included.